FR-300: Admin Features 2
FR-310: Question Management 3
FR-320: Conversation Management 8
FR-330: Image Management 11
FR-340: PDF Management 14
FR-350: Build Index Button 17
FR-360: Versioning Features 19
FR-370: Diseases Management 22
FR-380: Automated Testing 37
FR-390: Dashboard 45

FR-300: Admin Features

FR-310: Question Management

I. Functional Requirements

ID	Requirement	Description
FR-311	Admin Question View	Admin users must be able to view the list of all questions (which can be filtered), including all the details of specific questions if requested.
FR-312	Admin Question Update/Create	Admin users must be able to update existing questions and its details, as well as adding more questions to the knowledge base.
FR-313	Admin Question Batch Create	Admin users must be able to add questions to the knowledge base in batches from a .csv or .xlsx file.

II. Use-case Specification

UC-311. Admin Question View

Summary: This UC describes the process where the user admin can get the questions and their answers, as well as the reasoning in the knowledge base

Goal: To provide an interface for the admins to view the knowledge base.

Preconditions:

- The user has the admin rights.
- The user is currently in the `View Question` interface.

Postconditions:

- Success: The user can view all the questions and view specific questions with their details.
- Failure: The user is unable to view the questions due to a lack of admin rights or a network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describes what users might see when they are viewing the list of questions

Step	**Actor Action (User)**	**System Action (Frontend)**

1	The user enters the `View Question` interface.	GOTO: `/admin/questions`
		
		If the user has admin rights and fetches successfully:
		• Display the list of questions, limit, and offset by pages.
		• Options for the filter also appear:
		  ◦ Number of questions per page
		  ◦ Search for the question title
		  ◦ Filter by the topic of the question/user asked
		  ◦ Sort the questions by topic
		
		Else:
		• Return a screen that notifies the user that they are unauthorized.

2	The user searches for the specific questions and clicks on them.	Shows the question, the answer to that question, and the supporting points

UC-312. Admin Question Update/Create

Summary: This UC describes the process that the admin can add questions and answers to the knowledge base.

Goal: To provide an interface for the admins to update the knowledge base, thus keeping the knowledge base up-to-date.

Preconditions:

- The user has the admin rights.
- The user is currently in the `View Question` interface.

Postconditions:

- Success: The user can view all the questions and view specific questions with their details.
- Failure: The user is unable to view the questions due to a lack of admin rights or a network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describes what users might see when they are giving feedback.

This scenario mainly describes what users might see when they are querying the feedback from the database. We start from the

Step	**Actor Action (User)**	**System Action (Frontend)**

1	The user clicks the `Input Question` interface.	GOTO: `/admin/question-input`
		If the user has admin rights and fetches successfully:
		• Display the interface for adding questions to the interface
		• The input fields include:
		  ◦ Topic/subtopic
		  ◦ Question (English & Vietnamese)
		  ◦ Answer (English & Vietnamese)
		  ◦ Supporting Points (English & Vietnamese)
		• The user can also add follow-up questions.
		Else:
		• Return a screen that notifies the user that they are unauthorized.

2	The user clicks `Submit` after filling in all the fields	• If fields are filled correctly:
		  ◦ The question is added to the list of questions
		• Else:
		  ◦ Output error: that field is missing or wrongly filled.

UC-313. Admin Question Batch Create

Summary: This UC describes the process that the admin can add questions in a batch to the knowledge base.

Goal: To provide an interface for the admins to update the knowledge base, thus keeping the knowledge base up-to-date.

Preconditions:

- The user has the admin rights.
- The user is currently in the `Question Input` interface.

Postconditions:

- Success: The user can view the `Question Input` page.
- Failure: The user is unable to view the `Question Input` page due to a lack of admin rights or a network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describes what admin users see when they are importing a data .

Step	**Actor Action (User)**	**System Action (Frontend)**
1	The user clicks the `Question Input` interface.	GOTO: `/admin/question-input` If the user has admin rights and fetches successfully: • Display the interface for adding questions to the interface Else: • Return a screen that notifies the user that they are unauthorized.
1	The user clicks the `Question Input` interface.	GOTO: `/admin/question-input` If the user has admin rights and fetches successfully: • Display the interface for adding questions to the interface Else: • Return a screen that notifies the user that they are unauthorized.

2	The user clicks `Submit` after filling in all the fields	• If fields are filled correctly:
		  ◦ Question is added to the list of questions
		• Else:
		  ◦ Output error that field is missing or wrongly filled.

FR-320: Conversation Management

I. Functional Requirements

ID	Requirement	Description
FR-321	Admin Conversation View	Admin users must be able to view the list of all example conversations (which can be filtered), including all the details of specific questions if requested.
FR-322	Admin Conversation Update/Create	Admin users must be able to update existing conversations and its details, as well as adding more conversations to the knowledge base.

II. Use-case Specification

UC-321. Admin Conversation View/Update

Summary: This UC describe the process where the user admin can view all the example conversations in the knowledge base.

Goal: To provide an interface for the admins to view the example conversations in the knowledge base.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface.

Post-conditions:

- Success: The user can view all the conversations and view specific conversations with their details.
- Failure: The user is unable to view the conversations due to lack of admin rights or network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are viewing the list of questions

Step	**Actor Action (User)**	**System Action (Frontend)**
1	The user enters the `View`	GOTO: `/admin/conversations`

	`Conversation` interface.	If the user have admin rights and fetch successfully: • Display the list of questions, limit and offset by pages. • Options for filter also appear: ◦ Question in English/Vietnamese Else: • Return a screen which notifies the user that they are unauthorized.
2	The user search for the specific conversation and click it.	Shows the question, the answer of that question and the supporting points
3	The user clicks `Edit` or `Delete` button	If `Edit` button is clicked: • Edit interface is displayed for that conversation, the admin can edit it • Click `Submit` after that to complete updating the conversation If `Delete` button is clicked: • Delete that specific conversation.

UC-322. Admin Conversation Update/Create

Summary: This UC describe the process that the admin can add example conversations to the knowledge base.

Goal: To provide an interface for the admins to update the knowledge base, thus keeping the knowledge base up-to-date.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface.

Postconditions:

- Success: The user can add the new example conversation to the list of example conversations.
- Failure: The user is unable to add new conversations due to lack of admin rights or network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are giving feedback.

This scenario mainly describe what user might see when they are querying the feedbacks from the database. We start from the

Step	**Actor Action (User)**	**System Action (Frontend)**
1	The user clicks the `Input Conversation` interface.	GOTO: `/admin/conversation-input` If the user have admin rights and fetch successfully: • Display the interface for adding conversation to the interface • The user can click `Add Entry` to add new messages to the new conversation Else: • Return a screen which notifies the user that they are unauthorized.

2	The user clicks `Submit` after filling in all the fields	• If fields are filled correctly:
		  ◦ Conversation is added to the list of all questions
		• Else:
		  ◦ Output error that field is missing or wrongly filled.

FR-330: Image Management

I. Functional Requirements

ID	Requirement	Description
FR-331	Admin Domain Images View/Update	Admin users must be able to view the list of all domain images (which can be filtered), including all the details of specific images if requested. The images can be updated by authorized admins.
FR-332	Admin Domain Images Create	Admin users must be able to add more images to the knowledge base.

II. Use-case Specification

UC-331. Admin Domain Images View

Summary: This UC describe the process where the user admin can view all the images in the knowledge base and their details.

Goal: To provide an interface for the admins to view the images in the knowledge base to check for validity and export for downstream tasks.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface.

Post-conditions:

- Success: The user can view all the images with their details
- Failure: The user is unable to view the images due to lack of admin rights or network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are viewing the list of images

Step	Actor Action (User)	System Action (Frontend)

1	The user clicks the `Domain Images` button.	GOTO: `/admin/images`
		
		If the user have admin rights and fetch successfully:
		
		• Display the list of images, limit and offset by pages.
		• Options for filter also appear:
		  ◦ Images per page
		  ◦ Name of images
		
		Else:
		
		• Return a screen which notifies the user that they are unauthorized.

2	The user clicks on the `Edit` button in each image	GOTO: `/admin/images/{image_id}` The edit interface will show up and the user can modify existing information • Disease • English description • Vietnamese description • Source of the image
3	The user click `Submit`	The image will be modified according to the input information

UC-332. Admin Images Create

Summary: This UC describe the process that the admin can add images to the knowledge base.

Goal: To provide an interface for the admins to update the knowledge base, thus keeping the knowledge base up-to-date.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface.

Postconditions:

- Success: The user can add the new image to the list of images.
- Failure: The user is unable to add new images due to lack of admin rights or network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are uploading images

Step	**Actor Action (User)**	**System Action (Frontend)**

1	The user clicks the `Domain Images` button.	GOTO: `/admin/images`
		If the user have admin rights and fetch successfully:
		• Display the list of images, limit and offset by pages.
		• Options for filter also appear:
		  ◦ Images per page
		  ◦ Name of images
		Else:
		• Return a screen which notifies the user that they are unauthorized.

2	The user clicks `Add New Image`	GOTO: `/admin/image-input`
		• An interface for adding images will show up, with options for upload images and the following information:
		  ◦ Disease
		  ◦ English description
		  ◦ Vietnamese description
		  ◦ Source of the image

3	The user click `Submit`	The image will be uploaded to the knowledge base.

FR-340: PDF Management

I. Functional Requirements

ID	Requirement	Description
FR-341	Admin Document View	Admin users must be able to view the list of all uploaded documents in the knowledge base.
FR-342	Admin Document Create/Delete	Admin users must be able to add or delete documents in the knowledge base.

II. Use-case Specification

UC-341. Admin Document View

Summary: This UC describe the process where the user admin can view all the documents in the knowledge base.

Goal: To provide an interface for the admins to view the documents in the knowledge base to check for validity and export for downstream tasks.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface.

Post-conditions:

- Success: The user can view all the documents with their details
- Failure: The user is unable to view the documents due to lack of admin rights or network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are viewing the list of documents

Step	**Actor Action (User)**	**System Action (Frontend)**
1	The user clicks the `PDF Input` button.	GOTO: `/admin/pdf-input` If the user have admin rights and fetch successfully:

		• Display two tabs: `Upload files` and `View files` Else: • Return a screen which notifies the user that they are unauthorized.
2	The user clicks on the `View files` tab	The viewing interface will show up and the user can see the list of documents with • Name • Two buttons: `View` and `Delete`
3	The user click `View`	The document will display in another tab

UC-342. Admin Images Create

Summary: This UC describe the process that the admin can add documents to the knowledge base.

Goal: To provide an interface for the admins to update the knowledge base, thus keeping the knowledge base up-to-date.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface.

Postconditions:

- Success: The user can add the new documents to the list of documents .
- Failure: The user is unable to add new documents due to lack of admin rights or network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are uploading images

Step	**Actor Action (User)**	**System Action (Frontend)**

1	The user clicks the `PDF Input` button.	GOTO: `/admin/pdf-input` If the user have admin rights and fetch successfully: • Display two tabs: `Upload files` and `View files` Else: • Return a screen which notifies the user that they are unauthorized.
2	The user clicks `Upload files`, click the `Select a file` button and upload the file	The file will be uploaded to the file knowledge base.

FR-350: Build Index Button

I. Functional Requirements

ID	Requirement	Description
FR-351	Admin Build Index	Admin users must be able rebuild the vector database for the updated conversations in the knowledge base, in order for the chatbot to perform RAG with up-to-date information

II. Use-case Specification

UC-351. Admin Build Index

Summary: This UC describe the process where the user admin can manually rebuild the vector databse for the updated conversations in the knowledge base.

Goal: To provide an interface for the admins to rebuild the vector database based on the conversations in the knowledge base, so that the RAG performed by the chatbot is up-to-date.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface.

Post-conditions:

- Success: The user can rebuild the vector database successfully.
- Failure: The user is unable to rebuild the database due to lack of admin rights or network failure.

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are synchronizing the knowledge base.

Step	**Actor Action (User)**	**System Action (Frontend)**
1	The user clicks the `Sync now` button.	If the user have admin rights and fetch successfully: • When building index: Display the loading animation and `Synchronizing`

		`Database...` • After complete: Display a notification `Successfully sync knowledge base`

FR-360: Versioning Features

I. Functional Requirements

ID	Requirements
FR-361	Admin can CRUD different versions of a specific disease
FR-362	Admin can select a specific version of a specific disease, and chatbot only process the selected versions

II. Data Schema Validation

```typescript
// Updated version compared to FR-370
type DiseaseVersion = {
    diseaseId: string;
    versionId: string;
    versionName: string;
    createdAt: number;
    active: boolean;
    en: {
        name: string;
        description: string;
        possible_pig_types: string;
        mortality_rate: string;
        morbidity_rate: string;
        ... 11 more ...;
        prevention?: string | undefined;
    };
    vi: {
        name: string;
        description: string;
        possible_pig_types: string;
        mortality_rate: string;
        morbidity_rate: string;
        ... 11 more ...;
        prevention?: string | undefined;
    };
    updatedAt?: number | ... 1 more ... | undefined;
    base?: boolean | undefined;
}

type Disease = {
    diseaseId: string;
    versionId: string;
    codeName: string;
    versionName: string;
    createdAt: number;
    en: {
        name: string;
        description: string;
        possible_pig_types: string;
        mortality_rate: string;
        morbidity_rate: string;
        ... 11 more ...;
        prevention?: string | undefined;
    };
```

```
    vi: {
        name: string;
        description: string;
        possible_pig_types: string;
        mortality_rate: string;
        morbidity_rate: string;
        ... 11 more ...;
        prevention?: string | undefined;
    };
    updatedAt?: number | ... 1 more ... | undefined;
}
```

Two tables are created for this purpose.

- The `Disease` table stores the current version of the `diseaseId`, together with the `versionId` immutable and `versionName` mutable
- The `DiseaseVersion` table stores the versions of the diseases with `diseaseId`, `versionId` as key and `base` is the base version of the disease (cannot be deleted if we do not delete the whole disease)

III. Use case specification

FR-361: Admin can CRUD different versions of a specific disease

Summary: This UC describe the process that the admin can CRUD different versions of a specific disease.

Goal: To help admin manage different versions of the same database.

Preconditions:

- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin/disases` or `/admin/diseases/[diseaseId]`

Postconditions:

- Similar to FR-370
- By default, only the active version of each disease is presented in the dashboard.
- There is a dropdown allowing the user to select a code name, then only all the versions of that disease is displayed.

FR-362: Admin can select a specific version of a specific disease, and chatbot only process the selected versions

Summary: This UC describe the process that admin can select a specific version of a specific disease, and chatbot only process the selected versions.

Goal: To apply a version into chatbot.

Preconditions:

- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin/disases` or `/admin/diseases/[diseaseId]`

Postconditions:

- Similar to FR-370
- When code name is selected, then the apply button is presented on each version, allowing user to select that version.

FR-370: Diseases Management

Quick Navigation

I. Functional Requirements

II. Data Schema Validation

III. Use Case Specifications

- FR-370: Diseases Management | UC 376 01: Admin view disease list
- FR-370: Diseases Management | UC 376 02: Admin search disease by codename
- FR-370: Diseases Management | UC 376 03: Admin filter disease list
- FR-370: Diseases Management | UC 376 04: Admin sort disease list
- FR-370: Diseases Management | UC 376 05: Admin view disease details
- FR-370: Diseases Management | UC 376 06: Admin activate/deactivate disease
- FR-370: Diseases Management | UC 376 07: Admin switch disease version
- FR-370: Diseases Management | UC 376 08: Admin paginate disease list

I. Functional Requirements

ID	Requirements	Description

FR-371	Admin creates a disease document	Admin fills the following fields to create a diseases:
		ℹ Diseases exist in multiple **Forms**. Form-specific fields must be clustered. Admin is able to add (or delete) and modify form-specific fields.
		• Codename (required)
		• General Info:
		  ◦ Name (free text markdown)
		  ◦ Pig Types (search, toggle on/off to select/deselect from a list)
		• Disease stats:

		◦ Mortality Rate (range selection, %) ◦ Morbidity Rate (range selection, %) ◦ Fever (range selection, Celcius degree) • Symptoms: ◦ Free Text • Management: All free text markdown ◦ Vaccine ◦ Treatment ◦ Recommendation • Additional Information: Free Text ◦ Immediate Actions ◦ Differential Diagnosis Admin can switch between and edit Vietnamese and English version of the fields.
FR-372	Admin updates a disease document	
FR-373	Admin reads a disease document	Displays a detailed view of a disease, presenting all information (using the same fields as FR-371) in a Markdown rich text format. **Actions:** • Administrators can toggle between Vietnamese and English versions. • Switch between versions of the disease.

		• Activate/Deactivate a version. • Add new version. • Delete current version.
FR-374	Admin deletes a disease document	
FR-375	Admin manually integrates the disease data to chatbot	
FR-376	Admin can view a list of Diseases	Provide a list of diseases with each item showing brief information, including: • Codename • Version • Warning of detail incompleteness (if applicable) • Last updated time • Active Status The disease list is the **first** thing Admin sees when navigate to Disease Management feature. Allow Admin to **search** by: • Codename Allow Admin to **filter** by: • Version • Completeness status • Active Status Allow Admin to **sort** by: • Codename • Last updated **Actions:**

		• Click on an item: Navigates to the corresponding detail page. • Create Button: Navigates to create disease page. • Edit Button of a disease item: Navigates to corresponding edit page. • Activate/Deactivate a disease. • Switch Version of a disease via Dropdown (require Activate to make newly selected Version live) • Click Delete to quickly delete (deprecated) Pagination capabilities: • Admin can navigate to the next and previous page using dedicated controls. • Admin can jump directly to the first and last page. • Admin can navigate to a specific page by entering a page number.

II. Data schema validation

```
const DiseaseDetails = z.object({
  name: z.string(),
  description: z.string(),
  possible_pig_types: z.string(),
  mortality_rate: z.string(),
  morbidity_rate: z.string(),
  fever: z.string(),
  symptoms_digestive: z.string(),
  symptoms_respiratory: z.string(),
  symptoms_neurological: z.string(),
  symptoms_dermatology: z.string(),
  symptoms_reproductive: z.string(),
  vaccines: z.string(),
  recommendation: z.string(),
  differential_diagnosis: z.string(),
  immediate_actions: z.string(),
  treatment: z.string(),
  prevention: z.string(),
```

```
})

export const Disease = z.object({
  diseaseId: ObjectIdAsHexString,
  codeName: z.string(),
  createdAt: UnixTimestamp,
  updatedAt: UnixTimestamp.nullish(),
  en: DiseaseDetails,
  vi: DiseaseDetails
})
```

III. Use case specification

FR-371: Admin creates a disease document

Summary: This UC describes the process where the Admin creates a new disease document.

Goal: To help admin to provide new disease.

-

Postconditions:

Success:

- New disease document is created and stored successfully.
- All entered fields (including form-specific and multilingual content) are saved.
- Disease appears in the disease list.

Failure:

- Missing required fields (e.g., Codename): Display validation error.
- Server error: Display general error message.
- Invalid data input (e.g., out-of-range values): Display field-level validation errors.

-

Step	Actor Action (Admin)	System Action (Frontend/Backend)
1	Admin clicks “Create Disease”	System displays disease creation form
2	Admin enters **Codename (required)**	System validates required field
3	Admin selects language (Vietnamese / English)	System switches editable language context

4	Admin fills **General Info** (Name, Pig Types)	System saves input state dynamically
5	Admin fills **Disease Stats** (Mortality, Morbidity, Fever ranges)	System validates numeric ranges (%) and temperature
6	Admin fills **Symptoms (Free Text)**	System stores markdown content
7	Admin fills **Management** (Vaccine, Treatment, Recommendation)	System stores markdown content
8	Admin fills **Additional Information** (Immediate Actions, Differential Diagnosis)	System stores free text
9	Admin adds/modifies/deletes **Form-specific fields**	System dynamically updates field groups
10	Admin switches language and enters translated content	System maintains separate localized values
11	Admin clicks “Save”	System validates all fields
12	—	System persists disease document
13	—	System confirms success and redirects or displays success message

Preconditions:

- Admin is logged in.
- Admin is on the Disease Management page.
- “Create Disease” action is accessible.
- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin/disease-input`

Postconditions:

- Success: The disease document is successfully created, a success message is displayed.
- Failure: The document is not sent to the server successfully, a message notifying that an unsuccessful message is displayed.

FR-372: Admin reads a disease document

Summary: This UC describe the process that the admin can read a disease document.

Goal: To help admin checks the list of disease documents which have been created, ensuring the all the information is correct.

Preconditions:

- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin/disases` or `/admin/diseases/[diseaseId]`

Postconditions:

- Success: The details of diseases are displayed properly.
- Failure: Display an error viewer, showing that an error occurs during data fetching process.

FR-373: Admin updates a disease document

Summary: This UC describe the process that the admin can update a disease document.

Goal: To help admin to modify an existing disease.

Preconditions:

- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin/diseases/[diseaseId]/edit`

Postconditions:

- Success: The disease document is successfully created, a success message is displayed.
- Failure: The feedback is not sent to the server successfully, a message notifying that it is unsuccessfull is displayed.

FR-374: Admin deletes a disease document

Summary: This UC describe the process that the admin can delete a disease document.

Goal: To help admin to delete an existing unnecessary disease.

Preconditions:

- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin/diseases/[diseaseId]`

Postconditions:

- Success: The disease document is successfully deleted, a success message is displayed.
- Failure: The document fails to be deleted, a message notifying that an unsuccessful message is displayed.

FR-375: Admin deletes a disease document

Summary: This UC describe the process that the admin can integrate newest disease data to the chatbot.

Goal: To help admin to update the chatbot with the disease data.

Preconditions:

- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin`

Postconditions:

- Success: The after clicking the sync button, the embedding data is successfully created in the database.
- Failure: Display an unsuccessful message.

UC-376-01: Admin view disease list

Summary: This UC describe the process that the admin can view and interact with the list showing existing diseases.

Goal: Allow the Admin to see an overview of all diseases with key information.

Preconditions:

- The Admin is authenticated and logged in.
- The Admin is on the Admin dashboard.
- The Disease Management page is accessible.

Postconditions:

- Success:

- The system displays a paginated list of diseases.
- Each disease item includes: Codename, Version, Completeness Warning (if any), Last Updated Time.

- Failure:
  - Server error: Display general error message.
  - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin navigates to Disease Management page	System loads disease list. System displays list of diseases with required fields. System shows pagination controls

---

UC-376-02: Admin search disease by codename

Summary: This UC describes the process where the Admin searches for a disease by codename.

Goal: Allow the Admin to quickly locate a disease.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
  - The system filters and displays diseases matching the codename.
- Failure:
  - Server error: Display general error message.
  - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks search bar	Cursor appears
2	Admin types codename	System filters list in real-time System displays matching results

---

UC-376-03: Admin filter disease list

Summary: This UC describes filtering diseases by version and completeness status.

Goal: Allow Admin to narrow down the disease list.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
  - System displays filtered list based on selected criteria.
- Failure:
  - Server error: Display general error message.
  - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin selects filter criteria (Version / Completeness / Active Status)	System applies filters System updates disease list accordingly

---

UC-376-04: Admin sort disease list

Summary: This UC describes the process where the Admin sorts the disease list.

Goal: Allow Admin to organize diseases efficiently.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
  - Diseases are sorted based on selected criteria.
- Failure:
  - Server error: Display general error message.
  - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin selects sort criteria (Codename / Last Updated)	System sorts list
2	Admin selects ascending/descending	System updates order

UC-376-05: Admin access disease details

Summary: This UC describes the process where the Admin navigates to disease detail page.

Goal: Allow Admin to view full disease information.

Preconditions:

- Admin is logged in.

- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
  - Admin is redirected to the disease detail page.
- Failure:
  - Server error: Display general error message.
  - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks a disease item	System navigates to detail page

---

UC-376-06: Admin activate/deactivate disease

Summary: This UC describes the process where the Admin toggles disease active status.

Goal: Allow Admin to control disease availability.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
  - Disease status is updated.
- Failure:
  - Server error: Display general error message.
  - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks Activate/Deactivate	System updates status and refreshes UI. Activate/Deactivate button switches to Deactivate/Active.

UC-376-07: Admin switch disease version

Summary: This UC describes the process where the Admin toggles switches between disease versions.

Goal: Allow Admin to manage versions and activate a selected one.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.
- Disease has multiple versions.

Postconditions:

- Success:
  - Selected version becomes active after activation.
- Failure:
  - Server error: Display general error message.
  - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks on Versioning dropdown of the disease.	Dropdown shows: • List of versions • Current active version

2	Admin selects a version	System displays selected version
3	Admin clicks Activate	System sets version as active

UC-376-08: Admin paginate disease list

Summary: This UC describes the process where the Admin navigates through pages of the disease list.

Goal: Allow Admin to browse disease list efficiently.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.
- Disease list contains multiple pages.

Postconditions:

- Success:
  - Correct page is displayed.
- Failure:
  - Server error: Display general error message.
  - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks Next/Previous	System loads corresponding page in **no more than 300ms**. UI indicate the current page Admin is on.
2	Admin clicks First/Last	System loads corresponding pagein **no more than 300ms.**

		UI indicate the current page Admin is on.
3	Admin enters page number	System loads corresponding page in **no more than 300ms**. UI indicate the current page Admin is on.

IV. Mock GUI

https://gemini.google.com/share/e0e95907da66

V. Revision History

Versioning Scheme: MAJOR.MINOR.PATCH

Version	Date	Author	Changes Summary
1.0.0	-	-	Initial release
1.1.0	28/04/2026	BVBMinh	

FR-380: Automated Testing

I. Functional Requirements

ID	Requirement	Description
FR-381	Synthetic Profile Management	A synthetic profile must include: • Profile name • Basic farm information • Symptoms • Picture (optional) • **Label:** the target disease Admin must be able to view/create/edit/delete profiles. Admin must be able to: • filter by name/label • search by name/id. • sort by name/label Admin must be able to do bulk action. Admin must be able to view list of profile with brief description of them: • Id • Name • Label
FR-382	Automated Test Execution	A testing session should have the following settings: Pre-run: • Pick tested model • Synthetic Profile Selection • Number of repetitions • (Optional) AI anomaly detection • (Optional) AI accuracy report The result of the test should include: • IDs of the generated chat • Report of returned errors (with link to them)

		• (Optional) Anomaly Report • (Optional) Accuracy Report
FR-383	AI Conversation Engine	LLM will be used to answer the chatbot questions using information from: • Synthetic Profile • Dictionary • Chat History
FR-384	Test Suite Management	Admin manages test suites with CRUD operations. Admin creates or updates a test suite by configuring the following settings for a test suite: • Which model • Which mode • What Synthetic Profile (SP) to use (allow preview). • Number of workers (default 1)
FR-385	Admin views test suite results	Upon execution, the system must return a list of test cases included, presenting the following metrics for each case: • Disease • Flu (Fluency) • Eff (Efficiency) • Lat (Latency) • Turns • Acc (Accuracy) • Cost • Step
FR-386	Admin views full conversation details	Admin can select a specific test case to "Show full conversation". The detailed view must include: • Chat history (SP vs Chatbot). • Evolution/history of hypothesis (at every step). • Reasoning. • Scores (Accuracy, Fluency, etc.). • Lat breakdown table.

		• Chatbot's feedback.

FR-387	Admin downloads reports	Download
		• Report as .md
		• Raw json data
		Summary Report:
		• See doc farm2vets.com/app/chatbot/reports/eval-en
		• Download
		  ◦ Report as .md
		  ◦ Raw json data

II. Data schema validation

```
const TestCaseMetrics = z.object({
})

const ConversationDetails = z.object({
})

const TestCase = z.object({
  testCaseId: z.string(),
  syntheticProfileId: z.string(),
  metrics: TestCaseMetrics,
  conversation: ConversationDetails.optional(),
})

export const TestSuite = z.object({
  testSuiteId: ObjectIdAsHexString,
  model: z.string(),
  mode: z.string(),
  syntheticProfiles: z.array(z.string()),
  numWorkers: z.number().default(1),
  testCases: z.array(TestCase).optional(),
  createdAt: UnixTimestamp,
  updatedAt: UnixTimestamp.nullish(),
})
```

III. Use-case Specification

UC-381. Synthetic profile management

Summary: This UC describe the process where the user admin can view and edit the current existing synthetic profiles.

Goal: To provide an interface for the admins to manage the synthetic profiles for automated testing purposes.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface. `Admin`

Post-conditions:

- Success: The user can view all the synthetic profiles and edit those profiles
- Failure: The user is unable to view the synthetic profiles and/or unable to edit them due to network errors or insufficient permissions

---

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are viewing the list of questions

Step	**Actor Action (User)**	**System Action (Frontend)**

1	The user enters the `Voew Profiles` interface.	GOTO: `/admin/profiles`
		If the user has admin rights and fetch successfully:
		• Display the list of profiles, limit and offset by pages.
		  ◦ The profiles in the list should at least contain the metadata:
		  ▪ Name
		  ▪ Disease Label
		• Options for filter also appear:
		  ◦ By name/label (disease)
		• Options for sort also appear:
		  ◦ By name/label
		• Options for searching also appear:
		  ◦ By name/disease
		Else:
		• Return a screen which notifies the user that they are unauthorized.

2	The user filters, sorts and	Show the list of profiles according to the search parameters

	searches according to their preference.	
3a	The user clicks `View` button or `Delete` button	If `View` button is clicked: • GOTO `/admin/profiles/{profileId}` • Shows the information that is in the profile with editor available If `Delete` button is clicked: • Show a prompt to confirm the deletion action • If the user agrees, then `DELETE` the profile and return back to the user interface.

UC-382. Automated Test Execution

Summary: This UC describe the process that the admin can execute automated test execution for the user profiles.

Goal: To provide an interface for the admins to execute automatic test execution for testing the accuracy of the chatbot without having to manually entering the information to the chatbot.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface.

```
```

Postconditions:

- Success: The user can perform automated test execution and receives the results of the tests performed
- Failure: The user is unable to perform automated test execution due to network errors or insufficient permissions

---

Main Success Scenario (Basic Flow)

This scenario mainly describe what user might see when they are giving feedback.

This scenario mainly describe what user might see when they are querying the feedbacks from the database. We start from the

Step	**Actor Action (User)**	**System Action (Frontend)**
1	The user clicks the `Execute Tests` interface.	GOTO: `/admin/tests` If the user has admin rights and fetches successfully: • Display the interface to adjust parameters for tests: ◦ Choose synthetic profiles ◦ Choose number of repetitions Else: • Return a screen which notifies the user that they are unauthorized.
2	The user clicks `Execute` after filling in all the fields	If the user has admin rights and fetches successfully: • Starting to perform tests (Results can be updated in real-time) (Optional) • After the tests have been successfully performed, the user can receive all the data about the tests, including the IDs, the content and link to the tests Else: • Return a screen which notifies the user that they are unauthorized, or there are networks error.

UC-384. Test Suite Management

Summary: This UC describes the process where the Admin creates, reads, updates, or deletes test suites.

Goal: To help admin define parameters and manage collections of automated tests.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface, Automated Testing Console.

Postconditions:

- Success: The test suite configuration is successfully saved, updated, deleted, or displayed.
- Failure: The system displays a validation or server error message.

Main Success Scenario (Basic Flow)

Step	**Actor Action (Admin)**	**System Action (Frontend)**
1	Admin clicks "Create Test" or selects an existing test to update.	GOTO: `/link/to/` If the user has admin rights and fetches successfully: • Display the interface to adjust parameters for tests: ◦ Generative Model ◦ Mode ◦ Choose synthetic profiles ◦ Choose number of repetitions
2	Admin clicks "Save".	System persists the test suite document and displays a success message.

UC-385. View Test Suite Result

Summary: This UC describes the process where the Admin views the results of an executed test suite.

Goal: Allow the Admin to see an overview of all run test cases and their performance metrics.

Preconditions:

- The user has the admin rights.
- The user is currently in `Admin` interface, Automated Testing Console.
- The test suite has been successfully executed.

Postconditions:

- Success: The system returns and displays a list of all included test cases.
- Failure: The system displays a validation or server error message.

Main Success Scenario (Basic Flow)

Step	**Actor Action (Admin)**	**System Action (Frontend)**
1	Admin navigates to a completed Test Suite.	GOTO: `/link/to/` If the user has admin rights and fetches successfully: • Display the interface to adjust parameters for tests: ◦ Generative Model ◦ Mode ◦ Choose synthetic profiles ◦ Choose number of repetitions
2	Admin clicks "Save".	System persists the test suite document and displays a success message.

FR-390: Dashboard

Quick Navigation

I. Functional Requirements

II. Data Schema Validation

III. Use Case Specifications

I. Functional Requirements

ID	Requirements	Description

FR-391	Admin navigates via a side panel	The system must feature a persistent left sidebar containing navigation links. The links must be categorized as follows:
		• **Data Management**
		  ◦ Disease
		  ◦ Question
		  ◦ Conversation
		  ◦ Image
		  ◦ Resource
		  ◦ PDF Input
		• **Automated Testing**
		• **Overview Dashboard**
		• **Sync database**

FR-392	Admin syncs current data to chatbot	The Overview Dashboard must include a "Sync database" button that allows the Admin to quickly sync the current data to the live chatbot environment.
FR-393	Admin navigates via dashboard cards	The main dashboard view must feature clickable cards representing different modules or consoles. Clicking a card redirects the Admin directly to the

		corresponding management console.

II. Data schema validation

III. Use case specification

FR-391: Admin navigates via sidebar

Summary: This UC describes how the Admin uses the left sidebar to access different management modules.

Goal: Provide persistent access to all areas of the Admin Interface.

Preconditions:

- Admin is logged in.

Postconditions:

Success:

- New disease document is created and stored successfully.
- All entered fields (including form-specific and multilingual content) are saved.
- Disease appears in the disease list.

Failure:

- Missing required fields (e.g., Codename): Display validation error.
- Server error: Display general error message.
- Invalid data input (e.g., out-of-range values): Display field-level validation errors.

Step	Actor Action (Admin)	System Action (Frontend/Backend)

1	Admin expands the sidebar	System displays structured items:
		• **Data Management**
		  ◦ Disease
		  ◦ Question
		  ◦ Conver(sation)
		  ◦ Image
		  ◦ Resource

		◦ PDF Input • **Automated Testing** • **Overview Dashboard** • **Sync database**
2	Admin clicks a specific link	System routes the user to the corresponding page and highlights the active link in the sidebar.
3	Admin clicks the "Sync database" button.	System displays a loading state or confirmation modal. System completes the sync and displays a success toast notification.

FR-392: Admin uses dashboard cards

Summary: This UC describes how the Admin interacts with summary cards on the main dashboard view.

Goal: Provide quick visual shortcuts to frequently used consoles.

Preconditions:

- Admin is logged in.
- Admin is on the Overview Dashboard page.

Postconditions:

Success:

- Admin is redirected to the corresponding console.

Failure:

- Server error: Display general error message.

Step	Actor Action (Admin)	System Action (Frontend/Backend)
1	Admin views the main dashboard.	System renders clickable cards for various modules.
2	Admin clicks on a specific card.	System routes the user to the associated management page.